

Enunciado - Práctico Integrador 2026

Descripción General

Como práctico integrador se solicita la creación de un **Sistema de Gestión de Eventos y Entradas** (tipo Ticketek), donde se destacan dos componentes principales:

- **Backend:** desarrollado en Golang, que brindará todas las interfaces (API REST) necesarias para dar solución a los requerimientos del negocio.
 - **Frontend:** desarrollado en React, que implementará las vistas finales y consumirá los servicios provistos por el backend.
-

1. Control de Versiones

- El código fuente debe ser manejado como repositorio y debe estar subido y disponible en Github.
 - El desarrollo debe ser colaborativo y comprobable entre todos los 3 o 4 integrantes del grupo.
 - Debe emplearse el flujo de trabajo **GitFlow** (ramas `main`, `develop` y `feature-branches`).
-

2. Base de Datos

- Como fuente de datos se utilizará un esquema **MySQL**.
 - El modelo de datos deberá soportar, como mínimo, tres entidades principales:
 - Usuarios
 - Eventos
 - Entradas
 - El equipo tiene libertad para agregar entidades adicionales según las necesidades del dominio.
 - Se pueden agregar entidades secundarias o intermedias si el diseño lo requiere.
 - Las tablas deben crearse y mapearse estrictamente utilizando el ORM **GORM**, implementando correctamente:
 - claves foráneas
 - relaciones correspondientes en las estructuras de Go
-

3. Backend

El backend debe implementar una arquitectura **MVC (Modelo-Vista-Controlador)**.

Autenticación de usuarios

Desarrollar endpoints que permitan implementar un sistema de login que maneje la sesión del usuario mediante JWT.

Requisitos

- Soporte para dos tipos de usuarios:
 - Cliente
 - Administrador
 - El token debe expirar luego de un tiempo definido.
-

Funcionalidades para Cliente

A. Exploración del Catálogo de Eventos

Descripción

- Permitir visualizar el listado general de eventos disponibles.
- Implementar al menos un mecanismo de filtro.

Permisos

- No requiere validación de token.
-

B. Detalle Específico de un Evento

Descripción

- Obtener información completa de un evento específico mediante su ID.

Permisos

- No requiere validación de token.
-

C. Compra de Entrada

Descripción

- Procesar la adquisición de entradas.
- Contemplar validaciones de negocio.

Permisos

- Requiere validación de token.
-

D. Historial de Entradas ("Mis Entradas")

Descripción

- Retornar únicamente las entradas del usuario autenticado.

Permisos

- Requiere validación de token.
-

E. Cancelación de Compra

Descripción

- Procesar la anulación de tickets previamente adquiridos.
- Definir qué sucede con el cupo liberado.

Permisos

- Requiere validación de token.
-

F. Traspaso de Entrada

Descripción

- Transferir una entrada a otro usuario del sistema.

Permisos

- Requiere validación de token.
-

Funcionalidades para Administrador

A. Creación de Evento

Descripción

- Registrar nuevos eventos.
- Aplicar validaciones de consistencia.

Permisos

- Requiere validación de token.

B. Actualización de Evento

Descripción

- Modificar atributos de eventos existentes.

Permisos

- Requiere validación de token.
-

C. Cancelación de Evento

Descripción

- Eliminar eventos del catálogo.

Permisos

- Requiere validación de token.
-

D. Reportes y Métricas

Descripción

- Obtener métricas de ocupación o ventas.
- Ejemplos:
 - entradas vendidas vs capacidad total
 - listado de compradores

Permisos

- Requiere validación de token.
-

Seguridad

- Las operaciones sensibles deben protegerse con JWT firmado.
 - Las contraseñas no deben almacenarse en texto plano.
 - Debe utilizarse hashing:
 - MD5
 - SHA256
 - u otro mecanismo equivalente
-

4. Frontend

Vistas para Cliente

Catálogo de Eventos (Home)

Debe incluir:

- listado/grilla de eventos
 - título
 - horario
 - búsqueda
 - filtros
-

Detalle del Evento

Debe mostrar:

- descripción
 - foto
 - horario
 - duración
 - cupo
 - demás información relevante
-

Gestión de Compra

- Permitir comprar entradas.
 - Mostrar:
 - confirmación de éxito
 - manejo visual de errores
-

Mis Entradas

Debe permitir:

- visualizar historial
 - cancelar compras
 - transferir entradas
-

Vistas para Administrador

Panel de Gestión de Eventos

Interfaz protegida que permita:

- editar eventos
 - eliminar eventos
-

Formulario de Eventos

Debe permitir:

- crear eventos
- editar eventos

Incluyendo:

- título
 - fecha
 - horario
 - cupo
 - categoría
 - validaciones del lado cliente
-

Reportes y Métricas

Visualización de:

- entradas vendidas
 - cupo máximo
 - demás métricas enviadas por backend
-

5. Testing

Pruebas Unitarias

- Implementar tests en Golang.
 - Utilizar:
 - `testing`
 - `testify`
-

Pruebas de Integración

- Testear respuestas HTTP.
 - Utilizar `httptest`.
 - Validar:
 - status codes
 - casos exitosos
 - casos de error
-

Cobertura

Objetivo:

- 80% de cobertura
 - principalmente en:
 - servicios
 - controladores
-

6. DevOps

Dockerizar la solución completa utilizando:

- Dockerfile
- Docker Compose

Incluyendo:

- Frontend
 - Backend
 - Base de Datos
-

7. Consideraciones Técnicas y Arquitectura

La estructura del proyecto debe dividirse en:

- `backend`
- `frontend`

El backend debe organizarse respetando las siguientes responsabilidades:

clients

Clientes que consumen servicios externos.

controllers

Controladores HTTP.

dao

Acceso a datos.

domain

Entidades y reglas de negocio.

services

Lógica de negocio.

utils

Funciones reutilizables.

main.go

Punto de entrada de la aplicación.

8. Documentación

Debe existir un archivo `README.md` completo en la raíz del repositorio.

Contenido sugerido

Título y descripción

Explicación clara del sistema.

Capturas de pantalla

Demostraciones visuales.

Tabla de contenidos

Opcional.

Tecnologías utilizadas

Listado de frameworks y librerías.

Requisitos previos

Versiones y herramientas necesarias.

Instalación y uso

Pasos para:

- clonar
- instalar dependencias
- configurar variables
- ejecutar frontend/backend
- correr tests

Diagrama de Base de Datos

Debe:

- estar incrustado en README
- existir dentro de `/docs`

Decisiones de diseño

Justificar al menos dos decisiones técnicas importantes.

9. Bonus Track

Funcionalidad Extra (Obligatoria)

Cada grupo debe:

- proponer
- analizar
- implementar

una funcionalidad no incluida en los requerimientos base.

Debe:

- ser única por grupo
- anunciarse previamente
- enviarse antes del 22 de mayo

Testing Avanzado (Opcional)

Testing Frontend

Ejemplo:

- React Testing Library

Testing E2E

Ejemplo:

- Cypress
- Playwright

10. Condiciones de Regularidad

Modalidad

- Grupal

Fecha de entrega

- Entre el 12 y el 19 de junio.

Requisitos mínimos

- Backend y frontend funcionales
- Conexión a base de datos
- Registro y login
- Hash de contraseñas
- JWT básico
- Funcionalidades Cliente completas
- README inicial
- 40% de cobertura en servicios/controladores

No requerido para regularidad

- Roles y permisos avanzados
 - Funcionalidades Admin
 - Dockerización
 - Bonus Track
-

11. Condiciones de Examen Final

Modalidad

- Individual

Requisitos adicionales

- Autorización completa con JWT y roles
- Funcionalidades Admin completas
- Dockerización
- Bonus Track obligatorio
- 80% de cobertura
- Documentación final completa